

[家 / を 開発する / を API リファレンス / を キャッシュと状態 / を st.session_state](#)

セッション状態

セッション状態は、ユーザー セッションごとに再実行間で変数を共有する方法です。Streamlit は、状態を保存して保持する機能に加えて、コールバックを使用して状態を操作する機能も公開します。セッション状態は、内部のアプリ全体でも持続します マルチページアプリ。

Streamlit Developer Advocate Dr. によるこのセッションステートの基本チュートリアルビデオをご覧ください。始めるためにマリサ スミス:



Session State basics

Streamlit





見る

Session State で値を初期化します

Session State API はフィールドベースの API に従っており、これは Python 辞書に非常によく似ています:

パイソン



```
# Initialization
if 'key' not in st.session_state:
    st.session_state['key'] = 'value'
```

フォーラム Ask AI

```
# Session State also supports attribute based syntax
if 'key' not in st.session_state:
    st.session_state.key = 'value'
```

読んで更新します

Session State で項目の値を読み、に渡すことで表示します `st.write` :

パイソン



```
# Read
st.write(st.session_state.key)

# Outputs: value
```

Session State の項目を、次の値を割り当てて更新します:

パイソン



```
st.session_state.key = 'value2'      # Attribute API
st.session_state['key'] = 'value2'    # Dictionary like API
```

セッション状態にあるものについて興味がありますか? 使用 `st.write` または魔法:

パイソン



```
st.write(st.session_state)

# With magic:
st.session_state
```

Streamlit は、初期化されていない変数にアクセスされた場合に便利な例外をスローします:

パイソン



```
st.write(st.session_state['value'])
```

Throws an exception!

KeyError: 'st.session_state has no key "value". Did you forget to initialize it?'

Traceback:

```
File "/.local/share/lib/python3.8/site-packages/streamlit/script_runner.py", line 34
    exec(code, module.__dict__)
File "/home/app.py", line 3, in <module>
    st.write(st.session_state['value'])
File "/.local/share/lib/python3.8/site-packages/streamlit/state/session_state.py", 1
    return state[key]
File "/.local/share/lib/python3.8/site-packages/streamlit/state/session_state.py", 1
    raise KeyError(_missing_key_error_message(key))
```

項目を削除

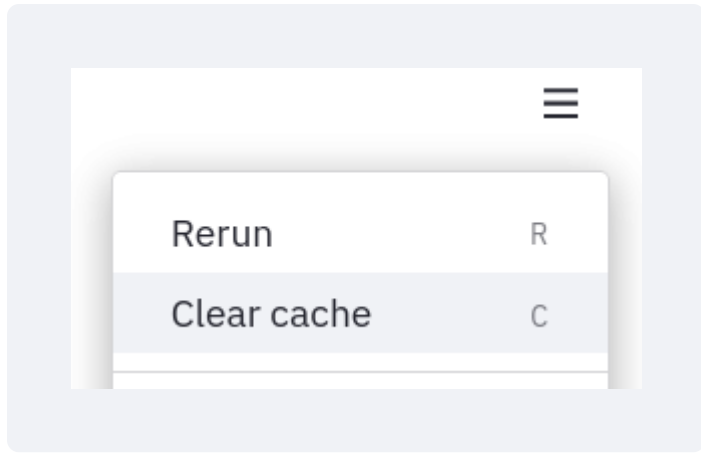
Session State で項目を削除する 構文を使用して、任意の Python 辞書内の項目を削除します:

パイソン

```
# Delete a single key-value pair
del st.session_state[key]

# Delete all the items in Session state
for key in st.session_state.keys():
    del st.session_state[key]
```

セッション状態は、「設定 → キャッシュのクリア」に進み、続いてアプリを再実行することによってもクリアできます。



セッション状態とウィジェット状態協会

キーを持つすべてのウィジェットは、Session Stateに自動的に追加されます:

パイソン

```
st.text_input("Your name", key="name")

# This exists now:
st.session_state.name
```

Session State を更新するには、コールバックを使用します

コールバックは、入力ウィジェットが変更されたときに呼び出される python 関数です。

執行の順序: に応じてSession stateを更新する場合 イベント, コールバック関数が最初に実行され、次にアプリが上から下に実行されます。

コールバックはパラメーターを使用してウィジェットで使用できます `on_change` (or `on_click`), `args`, および `kwargs`:

パラメータ

- **on_change** または **on_click** - コールバックとして使用する関数名
- **args** (タプル) - コールバック関数に渡される引数のリスト
- **クワーズ** (命令) - コールバック関数に渡される名前付き引数

をサポートするウィジェット `on_change` イベント:

- `st.checkbox`
- `st.color_picker`
- `st.date_input`
- `st.data_editor`
- `st.file_uploader`
- `st.multiselect`
- `st.number_input`
- `st.radio`
- `st.select_slider`
- `st.selectbox`
- `st.slider`
- `st.text_area`
- `st.text_input`
- `st.time_input`
- `st.toggle`

をサポートするウィジェット `on_click` イベント:

- `st.button`
- `st.download_button`
- `st.form_submit_button`

コールバックを追加するには、コールバック関数を定義します **上** ウィジェット宣言を作成し、ウィジェット経由でウィジェットに渡します `on_change` (or `on_click`) パラメータ。

フォームとコールバック

フォーム内のウィジェットには、Session State API 経由で値にアクセスして設定できます。 `st.form_submit_button` それに関連付けられたコールバックを持つことができます。送信ボタンをクリックするとコールバックが実行されます。例えば:

```
def form_callback():
    st.write(st.session_state.my_slider)
    st.write(st.session_state.my_checkbox)

with st.form(key='my_form'):
    slider_input = st.slider('My slider', 0, 10, 5, key='my_slider')
    checkbox_input = st.checkbox('Yes or No', key='my_checkbox')
    submit_button = st.form_submit_button(label='Submit', on_click=form_cal
```

Serializable Session State

シリアル化とは、オブジェクトまたはデータ構造を永続化して共有できる形式に変換し、データの元の構造を回復できるようにするプロセスを指します。Python の組み込み ピクルス モジュールは、python オブジェクトをバイト ストリーム ("pickling") にシリアル化し、ストリームをオブジェクト ("unpickling") に逆シリアル化します。

デフォルトでは、Streamlit's セッションの状態 オブジェクトの pickle シリアライズ可能性に関係なく、セッション期間中任意の Python オブジェクトを保持できます。このプロパティを使用すると、整数、浮動小数点数、複素数とブール値、データフレームなどの Python プリミティブを保存できます ラムダス 関数によって返されます。ただし、一部の実行環境では、Session State ですべてのデータをシリアル化する必要がある場合があるため、開発中に非互換性を検出すると便利な場合があります、または、実行環境が将来サポートを停止するとき。

そのために、Streamlit は次の機能を提供します `runner.enforceSerializableSessionState` 構成オプション に設定すると、それ `true` , session Stateでpickle-serializableオブジェクトのみを許可します。オプションを有効にするには、以下の内容でグローバルまたはプロジェクトコンフィグファイルを作成するか、コマンドラインフラグとして使用します:

```
# .streamlit/config.toml
[runner]
enforceSerializableSessionState = true
```

によって "ピクルスシリアル化可能", 「つまり、電話することです `pickle.dumps(obj)` を上げるべきではありません PicklingError 例外。config オプションが有効になっている場合、シリアル化不可能なデータをセッション状態に追加すると例外が発生するはずです。例えば、

```
import streamlit as st
```

```
def unserializable_data():  
    return lambda x: x
```

```
# 🚩 results in an exception when enforceSerializableSessionState is on  
st.session_state.unserializable = unserializable_data()
```

UnserializableSessionStateError: Cannot serialize the value (of type <class 'function'>) of 'unserializable' in st.session_state. Streamlit has been configured to use [pickle](#) to serialize session_state values. Please convert the value to a pickle-serializable type. To learn more about this behavior, see [our docs](#).

優先度 警告

とき `runner.enforceSerializableSessionState` に設定されています `true` , セッション状態は暗黙的に を使用します `pickle` 安全でないことが知られているモジュール。ピクル解除中に任意のコードを実行する悪意のあるピクルス データを構築する可能性があるため、セッション状態から保存および取得されるすべてのデータが信頼できることを確認します。安全でないモードで信頼できないソースから送信された可能性のあるデータや、改ざんされた可能性のあるデータは決してロードしないでください。信頼できるデータのみをロードします。

警告と制限

- Streamlit Session State は WebSocket 接続に関連付けられています。ユーザーがブラウザ タブを再ロードするか、Markdown リンクを使用して移動すると、WebSocket 接続と関連するセッション状態データがリセットされます。
- の だけ `st.form_submit_button` フォームにコールバックがあります。フォーム内の他のウィジェットにはコールバックが許可されていません。
- `on_change` そして `on_click` イベントは入力型ウィジェットでのみサポートされます。
- Session ステート API を介してウィジェットの値を変更すると、インスタンス化された後、許可されず、a が発生します `StreamlitAPIException` 。例えば:

```
slider = st.slider(
    label='My Slider', min_value=1,
    max_value=10, value=5, key='my_slider')

st.session_state.my_slider = 7

# Throws an exception!
```

My Slider



StreamlitAPIException: st.session_state.my_slider cannot be modified after the widget with key my_slider is instantiated.

Traceback:

```
File "/home/app.py", line 7, in <module>
    st.session_state.my_slider = 7
```

- Session State API を介してウィジェットの状態を設定し、を使用します `value` ウィジェット宣言のパラメータは推奨されず、最初の実行時に警告が表示されます。例えば:

```
st.session_state.my_slider = 7

slider = st.slider(
    label='Choose a Value', min_value=1,
    max_value=10, value=5, key='my_slider')
```


The widget with key "my_slider" was created with a default value but also had its value set via the Session State API.

Choose a Value



- ボタン型ウィジェットの状態を設定する: `st.button`, `st.download_button`, および `st.file_uploader` session State API 経由は許可されません。このようなタイプのウィジェットはデフォルトですぐに破棄されています。真 1 回の実行に対してのみ有効な状態。例えば:

パイソン

```
if 'my_button' not in st.session_state:
    st.session_state.my_button = True

st.button('My button', key='my_button')

# Throws an exception!
```

StreamlitAPIException: Values for the `st.button` and `st.file_uploader` widgets cannot be set using `st.session_state`.

Traceback:

```
File "/home/app.py", line 7, in <module>
    st.button('Submit', key='my_button')
```

フォーラム

まだ質問がありますか？

私たちのフォーラム役立つ情報と Streamlit の専門家がたくさんあります。